
Predicting Bank Term Deposit Subscriptions: A Machine Learning Approach to Customer Classification

Kaviya Appu Nawal Kazmi Ngoc Chau Ngo

Abstract

This paper studies binary customer classification for predicting whether a bank client will subscribe to a term deposit after a direct marketing campaign. The dataset contains 45,211 observations, 16 usable predictors after preprocessing, and a strongly imbalanced target distribution: 88.3% non-subscribers versus 11.7% subscribers. We compare Logistic Regression, Decision Tree, Random Forest, and LightGBM under a leakage-controlled workflow in which the train-test split is performed before fitting preprocessing operations. Imputation, signed log1p transformation, scaling, and encoding are fitted inside cross-validation folds. Class imbalance is addressed through stratified splitting, stratified cross-validation, class-weighted learning, and metrics beyond accuracy. LightGBM achieved the best cross-validation ROC-AUC and was selected as the final model. On the held-out test set, using the fixed 0.50 threshold used in the Colab final evaluation, it achieved 0.8463 accuracy, 0.4234 precision, 0.8667 recall, 0.5689 F1, 0.9269 ROC-AUC, and 0.5349 MCC. Results show that the model is useful for ranking likely subscribers. However, an ablation study confirms that the post-call feature duration strongly inflates performance, so a no-duration model is the more realistic pre-call deployment baseline.

Introduction

Banks use direct marketing campaigns to promote term deposits, but contacting every customer creates unnecessary cost, time burden, and customer fatigue. A machine learning model can support campaign planning by ranking customers according to their predicted likelihood of subscription before expensive outreach is expanded. The task is binary classification: predict whether the customer subscribes.

This project compares linear, tree-based, bagging-based, and boosting-based classifiers for the bank term deposit subscription task. The modeling problem has three practical complications. First, the target is highly imbalanced, so accuracy alone can be misleading. Second, several numerical features have long-tailed distributions, so preprocessing must reduce skew without losing information from negative balances. Third, hyperparameter search must be fair across models even though the models have different computational costs and different numbers of meaningful tuning parameters.

Our contribution is a reproducible, leakage-controlled comparison that explicitly reports how imbalance was handled, why signed log1p transformation was used, and why the search budget differs across models. The final model is selected using cross-validation only, and the held-out test set is used once for final evaluation.

Dataset and exploratory findings

The dataset is the Bank Marketing dataset associated with direct marketing campaigns of a Portuguese banking institution (UCI Machine Learning Repository, 2012; Moro et al., 2014). The data was collected from direct marketing campaigns conducted by a Portuguese retail bank between 2008

and 2013. The original data contains 45,211 records and 17 columns, including the binary target. After target encoding and feature engineering, the model uses 16 input features describing customer demographics, financial characteristics, and campaign history.

The feature groups and preprocessing strategy are summarized in Table 1. This separation is important because numerical, binary, and multi-category predictors require different transformations before being passed into the classifiers.

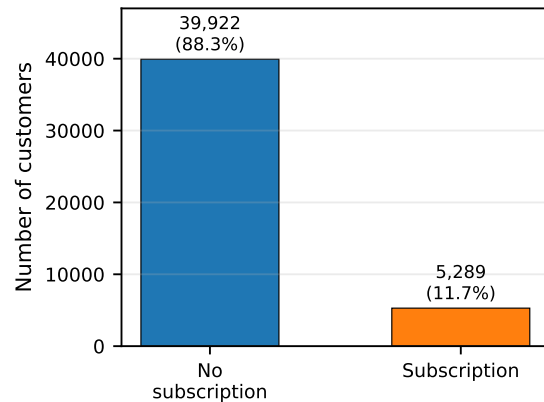


Figure 1: Target distribution. The positive class is only 11.7% of the dataset, so majority-class accuracy is not a sufficient evaluation criterion.

Figure 1 shows the key target imbalance: 39,922 observations are non-subscribers and 5,289 are subscribers, an approximately 8:1 imbalance ratio. A naive model that predicts non-subscription for all customers would achieve 88.3% accuracy while identifying no subscribers. Therefore, the project treats subscription detection as an imbal-

Table 1: Feature groups and preprocessing operations used inside the modeling pipeline.

Feature group	Features	Preprocessing strategy
Numerical	age, balance, duration, campaign, pdays	Median imputation, signed log1p transformation, standardization
Binary	default, housing, loan	Most-frequent imputation, one-hot encoding with one level dropped
Categorical	job, marital, education, contact, month, poutcome	Most-frequent imputation, one-hot encoding with unknown categories ignored
Pass-through	was_contacted	Retained as a binary indicator after row-wise feature engineering

anced classification problem rather than a simple accuracy-maximization task.

Exploratory analysis showed that duration has the strongest visual separation between subscribers and non-subscribers. However, duration is known only after or during the call, not before the campaign contact. The feature is therefore useful for measuring maximum predictive signal, but not fully realistic for pre-call targeting. This issue is tested later using an ablation study.

Leakage-controlled preprocessing

The workflow first encodes the target variable and performs row-wise feature engineering. The variable *pdays* records the number of days since a customer was last contacted in a previous campaign, using -1 as a sentinel value for customers who were never contacted. To avoid mixing two meanings in one variable, a binary feature *was_contacted* is created: it equals 1 when *pdays* is not -1 and 0 otherwise. The sentinel value -1 in *pdays* is then replaced with 0 so that absence of prior contact is not encoded as a misleading negative day count. The feature *previous* is dropped because 81.7% of its values are zero, making it close to a near-zero-variance predictor.

Figure 2 summarizes the pipeline. The data is split before any preprocessing object is fitted. The data uses an 80/20 stratified train-test split, with 36,168 training observations and 9,043 test observations. Within the training set, model selection uses 5-fold stratified cross-validation, giving an effective training/validation/test ratio of approximately 64/16/20. Stratification preserves the class ratio in both splits: the training split has 31,937 non-subscribers and 4,231 subscribers, while the test split has 7,985 non-subscribers and 1,058 subscribers.

Numerical features use median imputation, signed log1p transformation, and standardization. Binary features use most-frequent imputation and one-hot encoding with one level dropped. Multi-category features use most-frequent imputation and one-hot encoding with unknown categories ignored. Because these steps are inside a pipeline and column transformer, the scaler and encoders are fitted only on the training fold during cross-validation, reducing leakage risk.

Handling class imbalance

The imbalance is handled through the validation design, the learning objective, and the evaluation metrics.

Stratified split. The train-test split uses stratification. This prevents the minority class from being accidentally under-represented in either split and makes the held-out test set reflect the original class distribution.

Stratified cross-validation. Model comparison uses 5-fold stratified cross-validation with shuffling and a fixed random seed. Each fold preserves the positive-class ratio, which is important because precision, recall, F1, and ROC-AUC require enough positive examples in every validation fold.

Class-weighted learning. Models are trained with balanced class weights when supported. This increases the training penalty for errors on the minority subscriber class and reduces the penalty for errors on the majority class. This approach keeps the original data distribution intact; it does not oversample synthetic records or undersample the majority class.

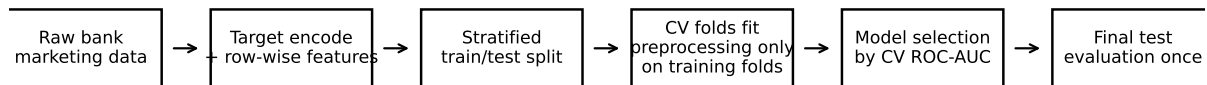
Metric choice. Accuracy is reported but not treated as sufficient. The main model-selection metric is ROC-AUC because it measures ranking quality across classification thresholds. F1, precision, recall, and Matthews correlation coefficient are also reported. Recall is business-critical because false negatives are missed subscribers. Precision matters because false positives represent unnecessary marketing contacts. MCC is useful because it summarizes confusion-matrix quality under imbalance.

Signed log1p transformation

Several numerical predictors are highly skewed. Balance, duration, and campaign contain long right tails and extreme values. Large outliers can dominate standardization and make linear-model fitting less stable. To reduce this effect, the numerical pipeline applies a signed log1p transformation:

$$x' = \text{sign}(x) \log(1 + |x|). \quad (1)$$

This is not “log base 1.” It is log1p, meaning log one plus the value. We used a signed version because bank balances can be negative. A standard logarithm cannot be applied directly to negative balances and is undefined at zero. The



Test set is locked before preprocessing is fitted; transformations are learned inside each training fold.

Figure 2: Leakage-controlled workflow. The test set is locked before preprocessing is fitted; preprocessing is learned inside each training fold during cross-validation.

signed version preserves the direction of debt versus positive balance, keeps zero as zero, and compresses large magnitudes. The goal is not to force perfect normality; the goal

is to reduce skew and stabilize feature scale before fitting models that are sensitive to magnitude.

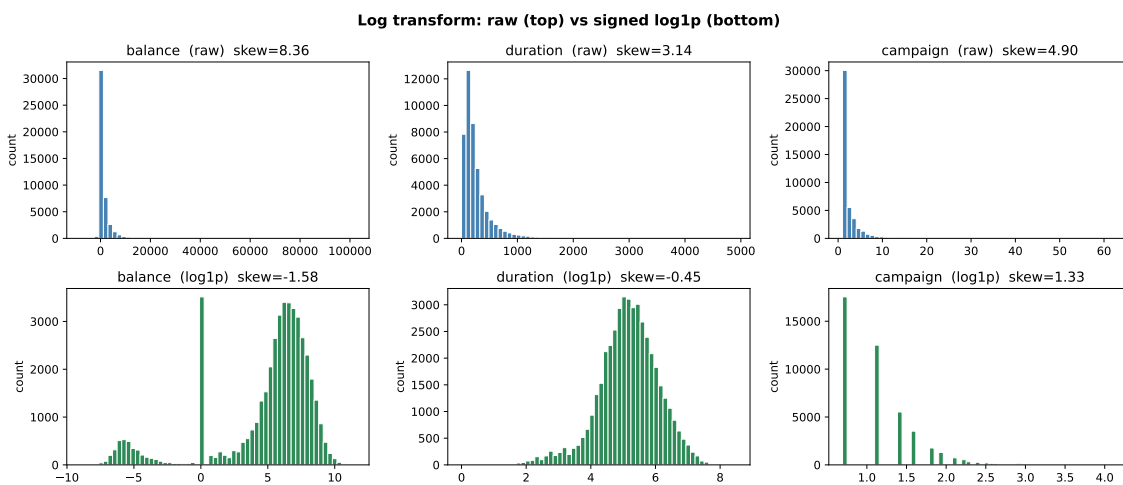


Figure 3: Effect of the signed log1p transformation on the three most skewed numerical features. The raw distributions are shown on the top row and the transformed distributions on the bottom row. Skewness decreases from 8.36 to -1.58 for balance, from 3.14 to -0.45 for duration, and from 4.90 to 1.33 for campaign.

Figure 3 illustrates the effect of this transformation. Before transformation, balance, duration, and campaign show strong right-skewness. After applying signed log1p, the distributions become more compact and numerically stable. This helps reduce the influence of extreme values during scaling and model fitting, especially for Logistic Regression.

Methods and evaluation

Each model was evaluated using stratified 5-fold cross-validation on the training set. The test set was excluded throughout model development and selection so that the final evaluation would not be biased by tuning decisions. Preprocessing steps were fitted within each cross-validation fold only on the training portion of that fold and then applied to the validation portion. This fold-level fitting prevents information leakage across splits.

ROC-AUC was the primary model-selection metric because it measures whether the model ranks likely subscribers

above unlikely subscribers across decision thresholds, which is well suited to the highly imbalanced target. Precision, recall, F1-score, and the train-CV gap in ROC-AUC were also used to evaluate practical usefulness and detect overfitting.

Logistic Regression baseline. Logistic Regression was used as the interpretable baseline because its coefficients provide a direct view of the direction and relative magnitude of each feature’s influence on the log odds of subscription. The model used the `lbfgs` solver, a maximum of 1,000 iterations, balanced class weights, and a fixed random seed. No hyperparameter search was performed because the goal of this model was to provide an interpretability anchor rather than to be the strongest competitive model. It was still wrapped in the same scikit-learn pipeline as the other models, including imputation, signed log1p transformation, standardization, and one-hot encoding.

The baseline achieved a CV ROC-AUC of 0.9088, recall of 0.8417, precision of 0.3834, F1-score of 0.5268, accuracy of 0.8231, and train-CV AUC gap of 0.0016. The small

train-CV gap indicates strong generalization, while the high recall and low precision reflect the effect of balanced class

weighting under class imbalance.

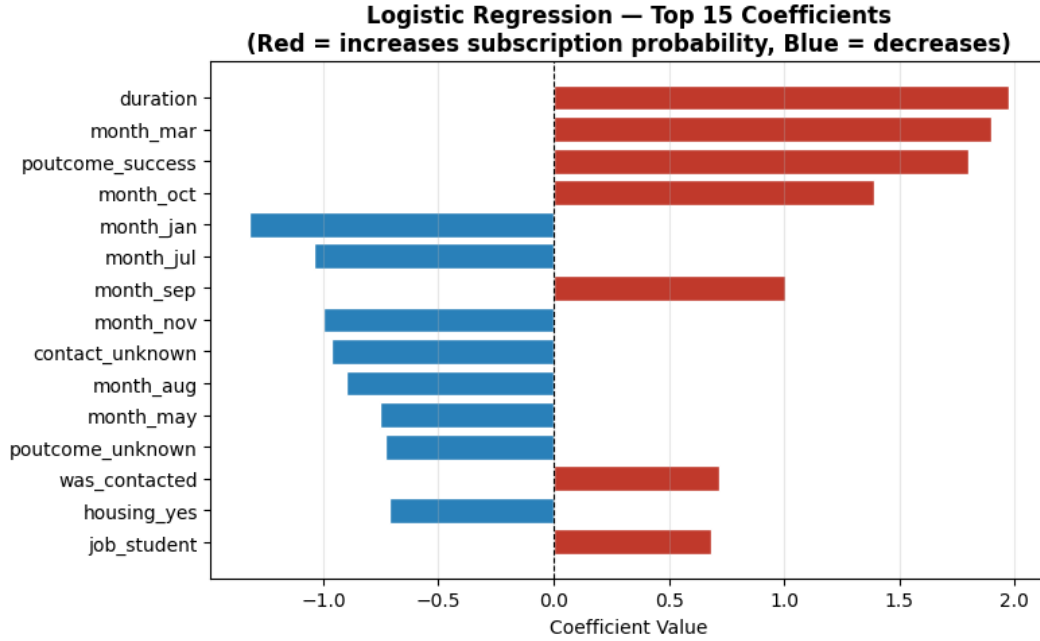


Figure 4: Logistic Regression coefficient analysis. Positive coefficients increase subscription probability, while negative coefficients decrease it. Duration is the strongest positive coefficient, but it is a post-call feature. Among pre-call signals, previous campaign success and contact month indicators are important.

Figure 4 shows that the strongest positive coefficient is *duration* (+1.9738), confirming the strong association between longer call duration and subscription. However, this is a post-call feature and cannot be used for pure pre-campaign targeting. Among more realistic pre-call features, *poutcome_success* (+1.7975) is a major positive signal, suggesting that customers with prior successful campaign outcomes are more likely to subscribe again. Month indicators also matter: March, October, and September have positive coefficients, while January, July, November, August, and May reduce the estimated subscription probability.

Decision Tree. The Decision Tree classifier was included as an interpretable nonlinear model. It can capture threshold effects and feature interactions without assuming a linear relationship between predictors and the log odds of the outcome. Because a single decision tree is prone to overfitting, depth and split-size constraints are important. The tree was tuned using `GridSearchCV` over the search space in Table 2. Candidate combinations were evaluated with 5-fold stratified cross-validation and ROC-AUC scoring.

The best Decision Tree used entropy, a maximum depth of 7, and a minimum split size of 50. It achieved a CV ROC-AUC of 0.8894, recall of 0.8615, precision of 0.3409, F1-score of 0.4880, accuracy of 0.7880, and train-CV AUC gap of 0.0195. This was the lowest ROC-AUC among the tested models, showing that a single tree could not capture the full predictive structure as well as ensemble methods. Its value

is mainly interpretability.

Random Forest. Random Forest was evaluated as a bagging-based ensemble. It trains many decision trees and averages their predicted probabilities, reducing the variance of a single tree. Because each candidate configuration trains many trees, `RandomizedSearchCV` with 6 sampled combinations was used instead of a large exhaustive grid. The search space is shown in Table 2.

The best Random Forest used 100 trees, maximum depth of 15, `sqrt` feature sampling, and a minimum leaf size of 5. It achieved CV ROC-AUC of 0.9235, recall of 0.8284, precision of 0.4467, F1-score of 0.5804, accuracy of 0.8598, and train-CV AUC gap of 0.0410. Random Forest produced the highest CV F1 score, but its larger AUC gap suggests more overfitting risk than LightGBM.

LightGBM. LightGBM was evaluated as the main boosting model. It builds an ensemble of decision trees sequentially, where each new tree focuses on correcting errors from the previous ensemble. LightGBM uses efficient gradient boosting with leaf-wise growth, which often performs well on structured tabular data (Ke et al., 2017). It was tuned with `RandomizedSearchCV` over 20 iterations because it has interacting hyperparameters and remained computationally efficient. The search space and the added regularization settings are shown in Table 2.

The best LightGBM search configuration used 100 estimators, maximum depth of 7, and learning rate 0.1. Additional regularization used `reg_alpha=0.1`, `reg_lambda=1.0`, and `min_child_samples=30`. L1 regularization encourages fewer nonzero feature contributions, L2 regularization penalizes large leaf weights, and the minimum child sample constraint prevents very small leaves from memorizing noise.

LightGBM achieved the highest CV ROC-AUC of 0.9275 and the highest recall of 0.8665. Its train-CV AUC gap of 0.0258 is lower than Random Forest's gap, suggesting better generalization after regularization. For that reason, LightGBM was selected as the final model for held-out test evaluation.

Table 2: Hyperparameter search spaces and regularization settings. Logistic Regression was fixed as the interpretable baseline and did not use hyperparameter search.

Model	Hyperparameter	Values searched
Decision Tree	criterion	{gini, entropy}
Decision Tree	max_depth	{3, 5, 7, 10, None}
Decision Tree	min_samples_split	{2, 10, 20, 50}
Random Forest	n_estimators	{100, 150}
Random Forest	max_depth	{5, 10, 15}
Random Forest	max_features	{sqrt, log2}
Random Forest	min_samples_leaf	{5, 10}
LightGBM	n_estimators	{100, 200, 300}
LightGBM	max_depth	{3, 5, 7}
LightGBM	learning_rate	{0.01, 0.1, 0.2}
LightGBM reg.	reg_alpha	0.1
LightGBM reg.	reg_lambda	1.0
LightGBM reg.	min_child_samples	30

This design uses the same validation protocol and selection metric for all models while allowing different search budgets according to computational cost. A full Decision Tree grid is cheap, while Random Forest requires fewer sampled configurations because each setting trains many trees. LightGBM receives a larger randomized search because its learning rate, depth, and number of estimators interact strongly and benefit from broader exploration. The test set is not used during any tuning step.

Cross-validation results

Table 3 reports training-set cross-validation results. LightGBM achieved the best mean CV ROC-AUC of 0.9275. Random Forest achieved the best mean CV F1 of 0.5804, but it also had a larger train-CV AUC gap than LightGBM.

Table 3: Cross-validation model comparison on the training set only.

Model	Acc.	Prec.	Rec.	F1	AUC	Gap
Logistic Regression	0.8231	0.3834	0.8417	0.5268	0.9088	0.0016
Decision Tree	0.7880	0.3409	0.8615	0.4880	0.8894	0.0195
Random Forest	0.8598	0.4467	0.8284	0.5804	0.9235	0.0410
LightGBM	0.8487	0.4276	0.8665	0.5726	0.9275	0.0258

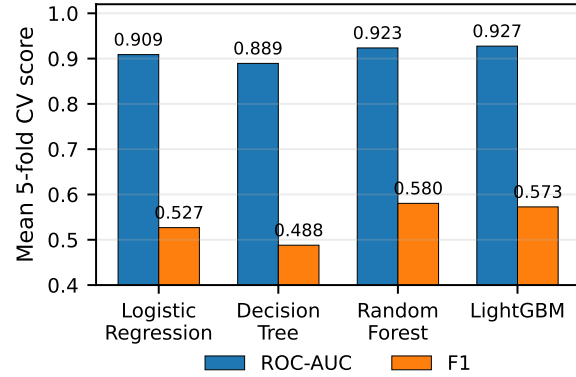


Figure 5: Cross-validation summary. LightGBM has the highest ROC-AUC, while Random Forest has the highest F1.

LightGBM was selected because the project prioritizes ranking likely subscribers under class imbalance. ROC-AUC is more aligned with ranking quality than one-threshold accuracy. The model also has a smaller AUC generalization gap than Random Forest. This makes LightGBM the strongest overall choice, even though Random Forest slightly leads on F1. Furthermore, Random Forest's larger AUC generalization gap, 0.0410 versus 0.0258 for LightGBM, suggests its cross-validation F1 estimate is slightly more optimistic, reinforcing LightGBM as the more reliable choice.

Final held-out evaluation

After model selection, the selected LightGBM pipeline is refit on the full training set and evaluated once on the held-out test set. The Colab final evaluation uses a fixed decision threshold of 0.50 for final class labels. Table 4 reports the final metrics. Predicted class labels and subscription probabilities were generated for all 9,043 held-out test observations and are provided in the supplementary notebook.

Table 4: Final LightGBM test-set performance.

Metric	Value
Accuracy	0.8463
Precision	0.4234
Recall	0.8667
F1 score	0.5689
ROC-AUC	0.9269
MCC	0.5349

The final model ranks subscribers well, with a test ROC-AUC of 0.9269. At the fixed 0.50 threshold, recall is 0.8667 and precision is 0.4234, giving an F1 score of 0.5689. This setting captures most actual subscribers, which is useful when missed subscribers are costly, but it also produces more false positives. The threshold should be adjusted in production if campaign cost, customer fatigue, or expected

revenue becomes the primary optimization target.

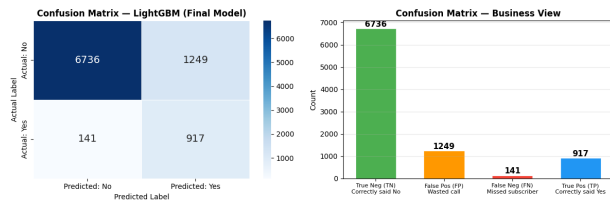


Figure 6: Final test confusion matrix for LightGBM at threshold 0.50. The left panel shows the standard confusion matrix; the right panel translates the same values into business terms: true negatives, false positives, false negatives, and true positives.

The confusion matrix in Figure 6 gives the business interpretation: 917 subscribers are correctly identified, 6,736 non-

subscribers are correctly ignored, 1,249 non-subscribers are unnecessarily flagged, and 141 subscribers are missed. This confirms that the model is recall-oriented at the fixed threshold: it captures most true subscribers, but the bank would still need threshold tuning if reducing unnecessary calls becomes more important than reducing missed subscribers.

Interpretation and deployment realism

LightGBM feature importance identifies duration, balance, age, days since previous contact, and campaign count as the most influential predictors. Logistic Regression coefficients also identify duration, March contact month, previous campaign success, and October contact month as strong positive signals.

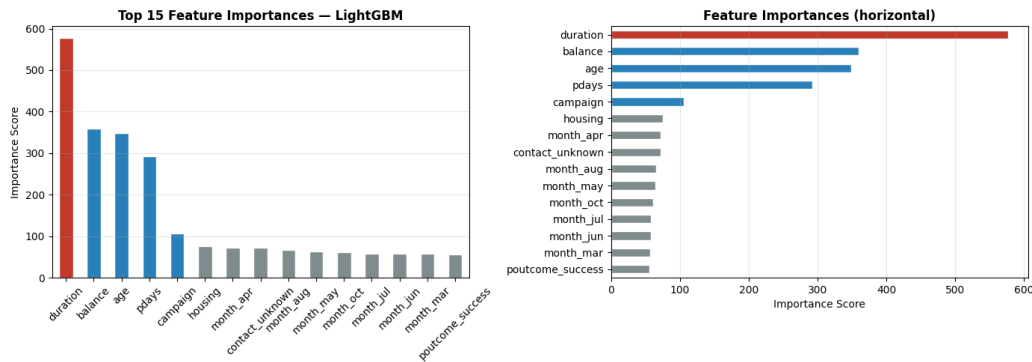


Figure 7: Top 15 LightGBM feature importances. The left panel shows vertical importances; the right panel gives the same ranking horizontally for readability. Duration dominates the model, creating a deployment concern because it is a post-call feature.

Feature importance analysis. The feature-importance plot shows that *duration* is the dominant predictor by a large margin. This means the model strongly relies on call length to separate subscribers from non-subscribers. That finding is useful analytically, but it also creates a deployment concern because duration is known only after the customer interaction begins. Among the more realistic pre-call variables, *balance*, *age*, *pdays*, and *campaign* carry the strongest signal. Month and contact-related variables also appear in the top rankings, suggesting that campaign timing and contact history affect subscription likelihood.

The most important practical issue is duration. It is predictive because longer calls often reflect customer engagement. However, it is not available before the bank makes the call. Therefore, a model that includes duration should not be presented as a pure pre-call targeting system. It is valid for analytical evaluation or during-call decision support, but not for pre-contact lead selection.

To quantify this issue, we reran LightGBM cross-validation without duration. Figure 8 compares the full-feature and no-duration settings.

The ablation shows a sharp drop: ROC-AUC decreases from 0.9275 to 0.7814, and F1 decreases from 0.5726 to 0.4302. This confirms that duration strongly inflates performance. The full model shows the maximum predictive signal in the dataset, while the no-duration model gives the more realistic pre-call baseline.

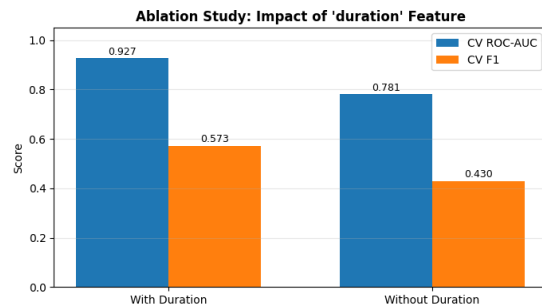


Figure 8: Ablation study. Removing duration substantially reduces ROC-AUC and F1, showing that pre-call deployment is harder than full-feature prediction.

Calibration and learning diagnostics

The calibration curve evaluates whether predicted probabilities match observed subscription frequencies. In Figure 9, the LightGBM curve lies mostly below the perfect-calibration diagonal. This indicates that the model's ranking quality is strong, but the raw predicted probabilities are not perfectly calibrated and should be treated as prioritization scores rather than exact subscription probabilities. If literal probability estimates are needed, future work should apply probability calibration methods such as Platt scaling, isotonic regression, or `CalibratedClassifierCV`.

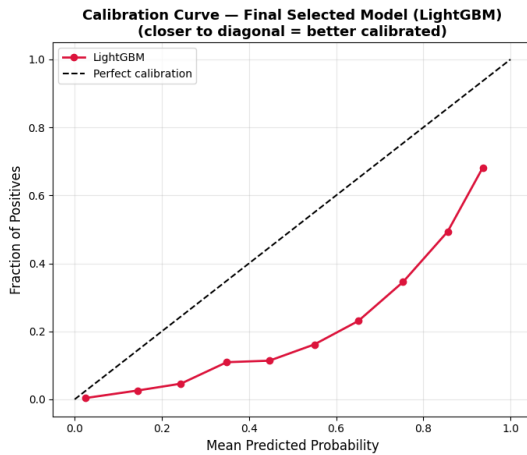


Figure 9: Calibration curve for the final LightGBM model. Points below the diagonal indicate over-confidence, while points closer to the diagonal indicate better probability calibration.

The learning curve in Figure 10 compares training ROC-AUC with cross-validation ROC-AUC as the training size increases. Validation performance improves and begins to stabilize near the full training size, suggesting that the model is learning meaningful structure rather than relying only on noise. The remaining train-validation gap indicates mild overfitting, but the gap is controlled by regularization and is consistent with the train-CV AUC gap reported for LightGBM.

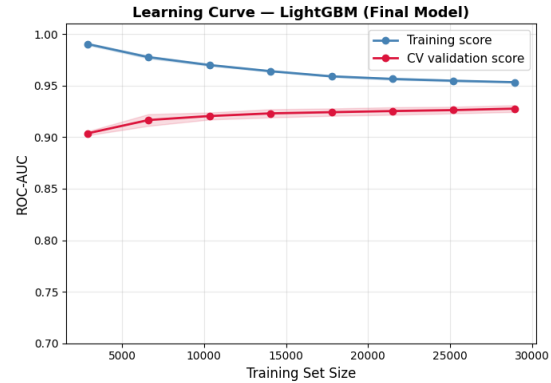


Figure 10: Learning curve for the final LightGBM model. The validation curve improves and stabilizes as more training data is used, while the gap from the training curve suggests mild but controlled overfitting.

Limitations and future work

This study has four limitations. First, the final threshold is fixed at 0.50 rather than optimized against an explicit campaign-cost, customer-annoyance, or expected-revenue function. Second, the full model includes duration, which is not available for pre-call targeting. Third, the evaluation uses a random stratified split; a time-based split would better simulate future campaigns. Fourth, the raw predicted probabilities are not fully calibrated, so a probability such as 0.80 should be treated as a ranking score rather than an exact subscription probability.

Future work should build a separate production-oriented pre-call model that excludes duration and any other post-contact information. Thresholds should be tuned using explicit business costs. Calibration methods such as Platt scaling or isotonic regression should be tested before using probabilities as literal risk estimates. A time-based validation strategy should also be used to estimate model performance under campaign drift.

Conclusion

This paper developed a leakage-controlled machine learning workflow for predicting bank term deposit subscriptions. Because the dataset is strongly imbalanced, the workflow uses stratified splitting, stratified cross-validation, balanced class weights, and evaluation metrics beyond accuracy. Signed log1p transformation is used to reduce skew while preserving negative balances. Different hyperparameter search budgets are justified by model complexity and computational cost, while all models are still compared under the same validation protocol.

LightGBM achieved the strongest CV ROC-AUC and strong held-out test performance, with 0.9269 test ROC-AUC, 0.5689 F1, and 0.5349 MCC at the fixed 0.50 threshold. The model is useful for ranking customers and reducing missed

subscribers, but operational deployment should tune the threshold against explicit business costs. The final deployment recommendation is to report both views: the full model shows maximum predictive signal, while the no-duration model is the honest baseline for pre-call campaign targeting.

Impact statement

The project supports more efficient marketing outreach by helping prioritize customers who are more likely to subscribe. A production system should still include fairness, privacy, and customer-contact safeguards, because predictive targeting can amplify biased outreach patterns or increase unwanted calls if used without governance.

References

- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 2017.
- Moro, S., Cortez, P., and Rita, P. A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62:22–31, 2014.
- Pedregosa, F. et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- UCI Machine Learning Repository. Bank Marketing Dataset. Center for Machine Learning and Intelligent Systems, 2012.